



CNN Dual Core Systolic Array with Maximum Throughput

Jupyter Jokers

Shan Ananth, Karthik Nandagudi Ravi Prakash, Nisha Kumar, Harini Sabapathy, Matthew Chan, Rana Singh



Motivation

This project is motivated by the need to design energy-efficient, reconfigurable systolic array accelerators that can adapt precision (2-bit/4-bit) and dataflow (weight-stationary/output-stationary) to diverse deep learning workloads, achieve high throughput and flexibility while maintaining accuracy, and validate these architectural innovations through practical FPGA mapping to bridge theory with real-world AI hardware deployment. In our work we have used the 2-D Systolic Array to accelerate VGGNet Model on CIFAR-10 dataset.

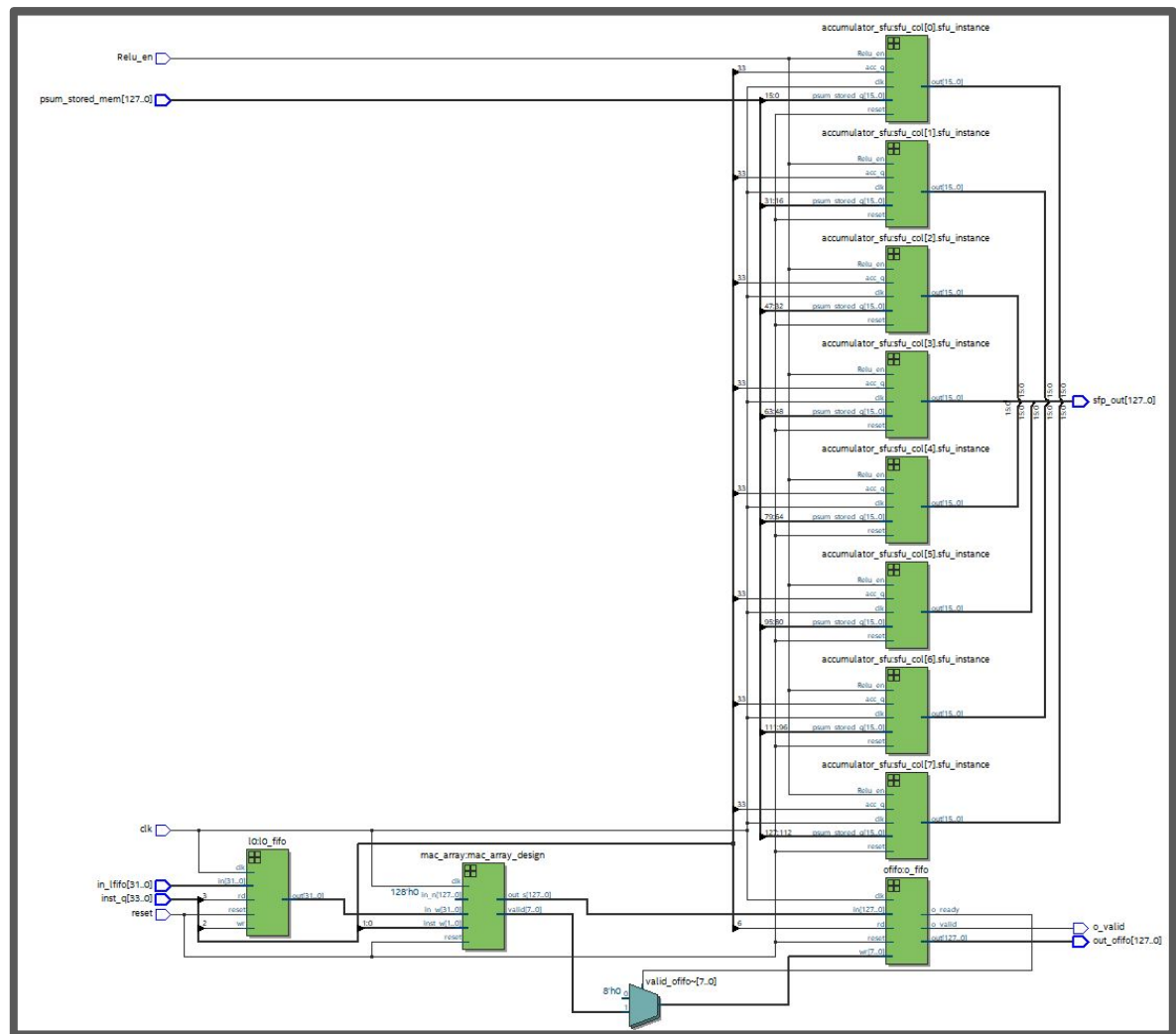


Figure 1 : RTL Netlist of Vanilla Design

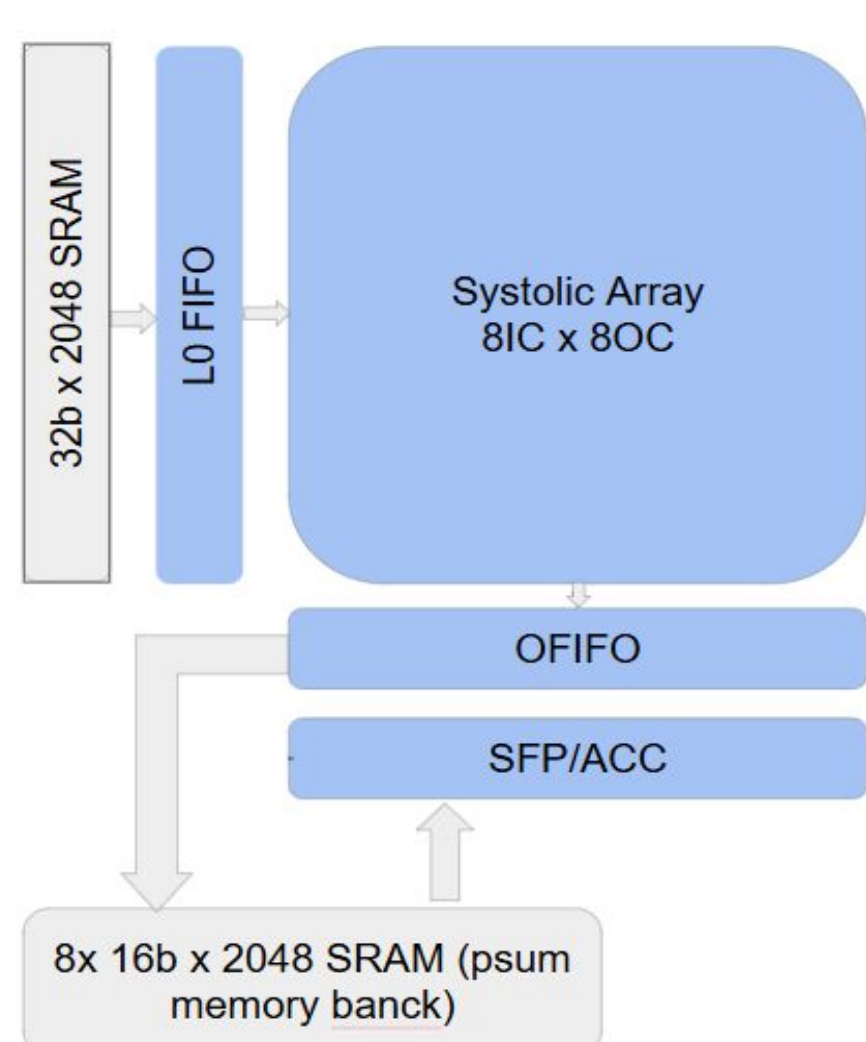


Figure 2: Vanilla design

Mapping on FPGA (Cyclone IV GX)	Vanilla Version
Total Operation per cycle	128
Frequency	124.5 MHz
Resource Utilization	Combinational ALUTs - 12074 Dedicated Logic Registers - 12098
Dynamic Power	35.08 mW
TOPs	0.015936
TOPs/W	0.45427

	VGG16 4-bit input 4-bit weight	VGG16 2-bit input 4-bit weight
Accuracy	92.02%	90.27%
Quantization Error	2.73e-06	7.33e-13

OPTIMIZED SOFTWARE ALPHAS

Structured Pruning

Through structured pruning we can greatly reduce the model size. If further implement structured sparsity in the hardware, it allows us to skip entire row/column in our MAC array, greatly reducing power consumption.

Sparsity : 0.4998
Estimated Memory Bandwidth Reduction: 49.98%

	VGG16 4-bit Unpruned Model	VGG16 4-bit Structured Pruned Model
Accuracy	92.02%	90.4%
Model Size	5.90 MB	2.95 MB

5 Optimized Hardware Alphas

1. Optimized testbench or Control Logic

For an $M \times M$ matrix, each kernel kij needs to be multiplied only by $(M-K+1) \times (M-K+1)$ activations. So we feed only the required activations for a kij efficiently. This gives a speedup of $(M \times M) / ((M-K+1) \times (M-K+1))$; for the M value of 6 and K value of 3 we get **2.25x speedup** and output memory area savings of $(K-1) \times OC$.

2. Memory Double Buffering & Parallel Sum Accumulation

- Output buffer:** Two single-port SRAMs, each with **8 banks for 8 output channels (OC)**, organized in a **ping-pong** fashion.
- Parallel read-write:**
 - In each cycle, **one SRAM is read** while the **other is written**, enabling **continuous accumulation** without stalls.
- Throughput benefit:**
 - Achieves up to **1.8x speedup** vs. a baseline that waits for *all* kernels' partial sums before accumulating.
- Area efficiency:**
 - 78% memory area reduction** by **eliminating per-kernel psum storage** and directly keeping only accumulated outputs.

Example ($K2 \rightarrow K3$ flow):

- For kernel **K2**: read psum from **Mem1**, accumulate, and write updated value to **Mem2**.
 - For kernel **K3**: read from **Mem2**, accumulate, and write back to **Mem1**.
 - This alternating pattern continues across kernels, maintaining **high utilization** of both SRAMs.
- ### 3. Unstructured Sparsity aware gating (in progress)
- Skip MAC Operation**
 - Since there are many zeros in activations(due to Relu, pruning in software) and weights(due to pruning) we can **save dynamic power** by skipping MAC operation and forwarding the input psum to output psum of the MAC tile whenever we have either of the activations or weights zero.
 - Skip Memory access**
 - If calculated psum is zero then avoid accessing memory as its waste of energy.

4. Interleaved & Pipelined Weight Loading (In-progress)

- Streaming path:** Weights are fetched from **memory** $\rightarrow$ **FIFO** $\rightarrow$ **accelerator core** in a **pipelined**, interleaved fashion.
- Overlap compute & load:**
 - While **partial sums (psums)** of the **current kernel** are being computed and streamed out, **Weights of the next-in-line kernel** are **preloaded in parallel** through the FIFO.
 - Adds a **dedicated weight FIFO(in WS adding IFIFO)** to decouple memory latency from the core's compute pipeline.
- Performance benefit:**
 - Hides weight-loading latency and **saves ≈ 70 clock cycles** compared to the baseline, where **weight loading only starts after** psums for a given kernel have fully completed.

5. Automated control logic in hardware

- With few external control in testbench like start, stop, Kernel size and Activation size the hardware computes and finally gives the output.

Alpha: SIMD 2/4bit Activation Dual-Core Processor with on-the-fly accumulation & dual-bank output SRAM

Using 16x8(2-bit)/8x8(4-bit) switchable variation, we tile 2 corelets horizontally to enable 16x16(2-bit) and 8x16(4-bit), allowing us to adapt our process to different layers / models. This change is expected to double our throughput.

Kernel writing is done sequentially (corelet 0 then 1) due to our limitation of having 1 input SRAM. Activation, execution, and accumulation are all done in parallel as activations are identical for horizontal tiles, and there are dedicated SRAM output sets for each corelet. Extracting final output, which should be 16x16b, is done sequentially as we only have one 8x16b SFP output bus from core.

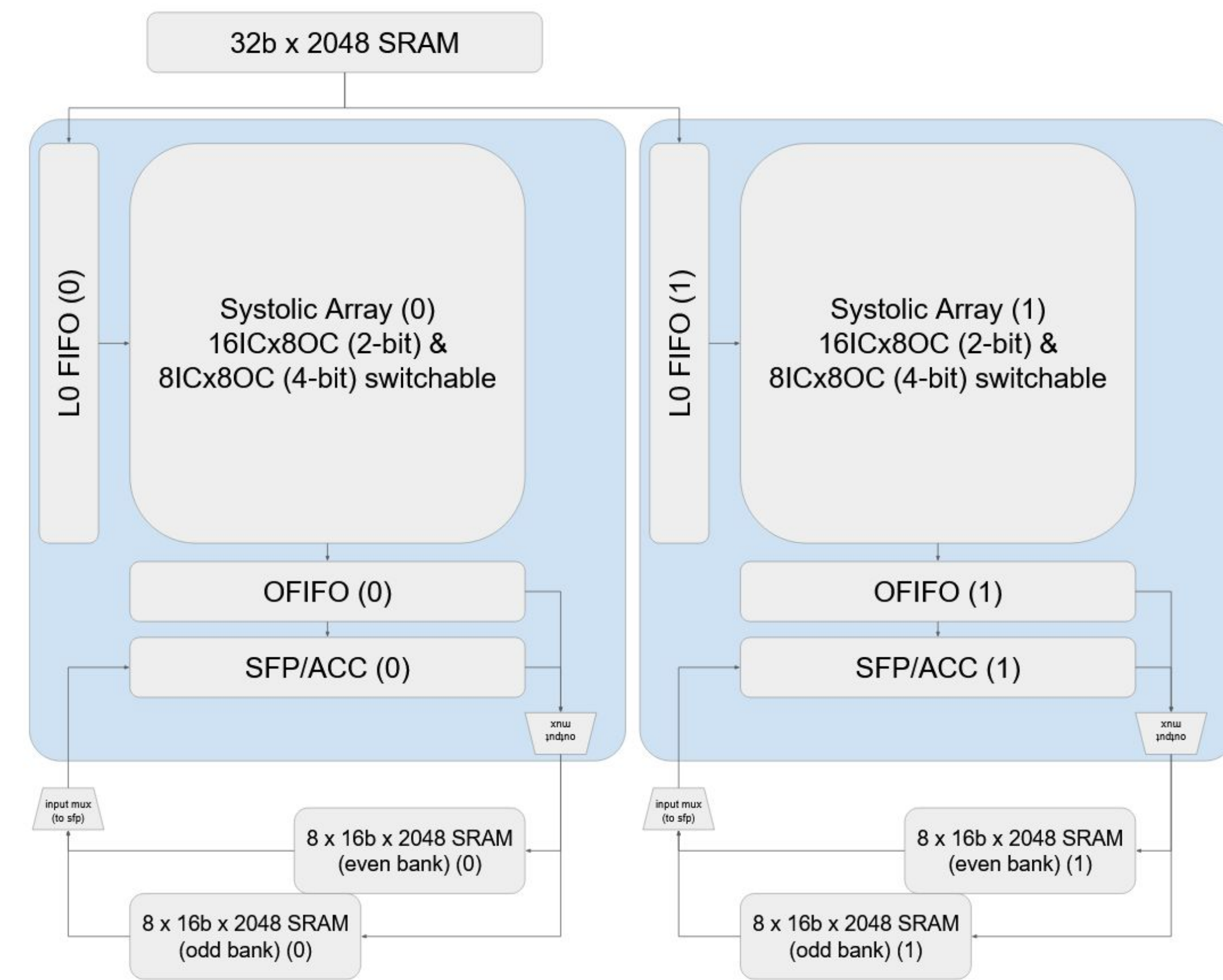


Figure 3: Dual-corelet SIMD 2/4 bit with dual-bank

```
[27] ✓ Os  output = [[ ] for _ in range(K_square)]
for i in range(K_square):
    h=0
    i_new = i
    mod_i = 0
    if i > K-1:
        mod_i = i // K
        h=mod_i*M
        i_new = i % K
        start = h + i_new
    for j in range(M-K+1):
        idx = start + j*M
        end = idx + M - K + 1
        while idx < end:
            output[i].append(v[idx])
            idx=idx+1

[28] ✓ Os  print(len(output))
clean_list = [int(x) for x in output[0]]
print(clean_list)

... 9
[0, 1, 2, 3, 6, 7, 8, 9, 12, 13, 14, 15, 18, 19, 20, 21]
```

Figure 4: Optimized selective activation feeding logic for specific kij